Local Smoothing for Mexican Hat Function

Qi Pan

October 2025

1 INTRODUCTION

Regression is a fundamental machine learning approach used to predict and explain the relationship between independent variables and a response variable. However, assuming a purely global linear relationship between predictors and response does not always hold for all datasets. In many real-world cases, the underlying relationships are nonlinear. In such situations, local smoothing methods provide an effective alternative by fitting the relationship between x and y based on local information rather than a single global model.

The goal of this homework is to study the statistical properties and computational challenges of several local smoothing methods, including LOESS, Nadaraya–Watson (NW) kernel smoothing, and spline smoothing. LOESS (Locally Estimated Scatterplot Smoothing) is a nonparametric technique that combines the idea of k-nearest neighbors with weighted linear regression. It works well for low-dimensional datasets and can flexibly model complex relationships, but it is computationally expensive and requires sufficient data density for stable results.

Kernel smoothing is another widely used local regression approach. In this method, the fitted value at each point is obtained as a weighted average of nearby observations, where the weights are determined by a kernel function—typically giving higher weights to points closer to the target. The degree of smoothness is controlled by the bandwidth parameter.

The Nadaraya–Watson estimator is a specific form of kernel smoothing that performs a local constant fit—that is, instead of fitting a local line (as in LOESS), it estimates the response at a point as the weighted average of nearby responses. While simpler and computationally efficient, it can introduce bias near boundaries or in regions with sparse data.

Spline smoothing is a third local smoothing technique. In spline methods, there are two main scenarios: interpolation and smoothing. Interpolation forces the fitted function to pass exactly through all data points, which often leads to severe overfitting. To address this issue, smoothing splines introduce a tuning parameter λ that controls the trade-off between goodness of fit and smoothness.

A spline is essentially a piecewise polynomial function, most commonly a cubic spline, that is joined smoothly at certain points called knots. The parameter λ acts as a roughness penalty, discouraging excessive curvature in the fitted function. This approach can be viewed as a special case of regularization. In practice, the optimal value of λ is often selected through cross-validation. A larger λ produces a smoother fit, while a smaller λ allows the model to follow the data more closely.

Understanding and comparing smoothing methods such as LOESS, Nadaraya–Watson (NW) kernel smoothing, and spline smoothing is important because these techniques are fundamental tools for uncovering underlying trends in noisy data. They help transform scattered observations into meaningful patterns without assuming a strict parametric form, making them highly valuable in real-world applications where data relationships are often complex and nonlinear. This analysis deepens our understanding of how different smoothing approaches balance bias and variance, adapt to local structures, and generalize across datasets.

2 EXPLORATORY DATA ANALYSIS (EDA)

In this homework, we simulate the data set consisting of n = 101 observations from the additive noise model $Y_i = f(x_i) + \epsilon_i$ with the famous Mexican hat function $f(x) = (1 - x^2)exp(-0.5x^2), st. -2\pi \leqslant x \leqslant 2\pi$. The error term $\epsilon_1, ..., \epsilon_n$ are independent and identically distributed (iid) with $N(0, 0.2^2)$. In the first experiment, the x_i values are evenly spaced within the interval $[-2\pi, 2\pi]$, resulting in a distance of approximately 0.1257 between consecutive points. In the second experiment, the x_i values are non-equidistant and randomly generated using random seed 79 in R. The scatter plots for both experiments are presented below, where the noise is generated from 1 run using random seed 7406.

3 METHODOLOGY

In the first experiment, we conduct a Monte Carlo simulation with m=1000 iterations to generate 1000 datasets, denoted as D_j for j=1,2,. . . ,m, each containing pairs (x_i, Y_i). Here, Y_i includes the additive noise component. For each dataset D_j, we compute three different types of local smoothing estimates at every point x_i: LOESS (with span = 0.75) by *loess()* function , Nadaraya–Watson (NW) kernel smoothing using a Gaussian kernel with bandwidth = 0.2 by *ksmooth()* function, and spline smoothing with the default tuning parameter by *smooth.spline()* function. All the functions are in R language. At each x_i, and for each smoothing method,

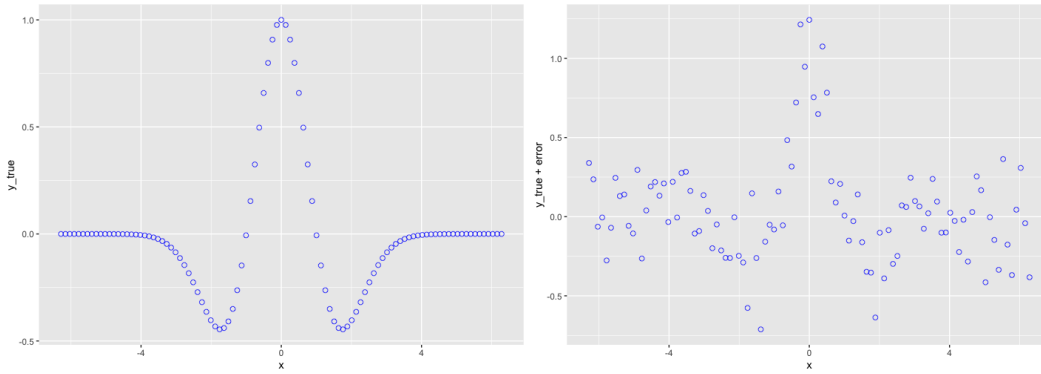

Figure 1—scatter plot of equidistant points without noise(left) and with noise(right)

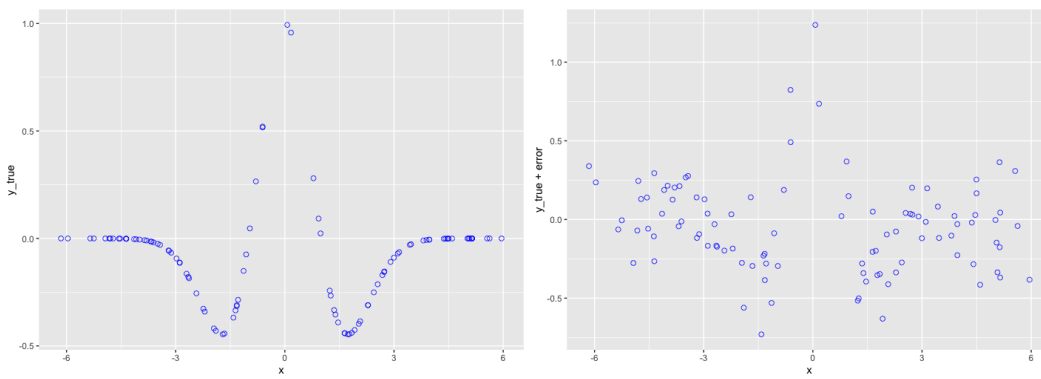

Figure 2—scatter plot of non-equidistant points without noise(left) and with noise(right)

we calculate the empirical bias, empirical variance, and empirical mean squared error (MSE) based on the 1000 Monte Carlo simulations, defined as follows:

$$\hat{\text{Bias}}\{f(x_i)\} = \bar{f}_m(x_i) - f(x_i), \qquad \text{s.t.} \bar{f}_m(x_i) = \frac{1}{m}\sum_{j=1}^{m} \hat{f}^{(j)}(x_i)$$

$$\hat{\text{Var}}\{f(x_i)\} = \frac{1}{m}\sum_{j=1}^{m} (\hat{f}^{(j)}(x_i) - \bar{f}_m(x_i))^2$$

$$\hat{\text{MSE}}\{f(x_i)\} = \frac{1}{m}\sum_{j=1}^{m} (\hat{f}^{(j)}(x_i) - f(x_i))^2$$

Note that we purposely use the coefficient $\frac{1}{m}$ (instead of the standard $\frac{1}{m-1}$) in the definition of empirical variance, so that the well-known relationship MSE =

$\mathtt{Bias}^2 + \mathtt{Var}$ holds for the empirical version as well. After computing all the values, we plot the mean fitted curve and the corresponding empirical bias, variance, and MSE against xi for all three local smoothing estimators: LOESS, Nadaraya–Watson (NW) kernel smoothing, and spline smoothing.

In the second experiment, we again perform 1000 Monte Carlo simulations to generate datasets. However, this time the xi values are non-equidistant, resulting in regions where the xi values are more densely distributed and others where they are relatively sparse. We continue to use the same three local smoothing methods—LOESS, Nadaraya–Watson kernel smoothing, and spline smoothing—but with slightly different tuning parameters: the span for LOESS is set to 0.3365, the bandwidth for NW kernel smoothing remains 0.2, and the smoothing parameter for spline smoothing (`spar`) is 0.7163. The resulting fitted curves and error metrics are presented in the Results section.

4 RESULT

4.1 Analyze the results of Experiment 1

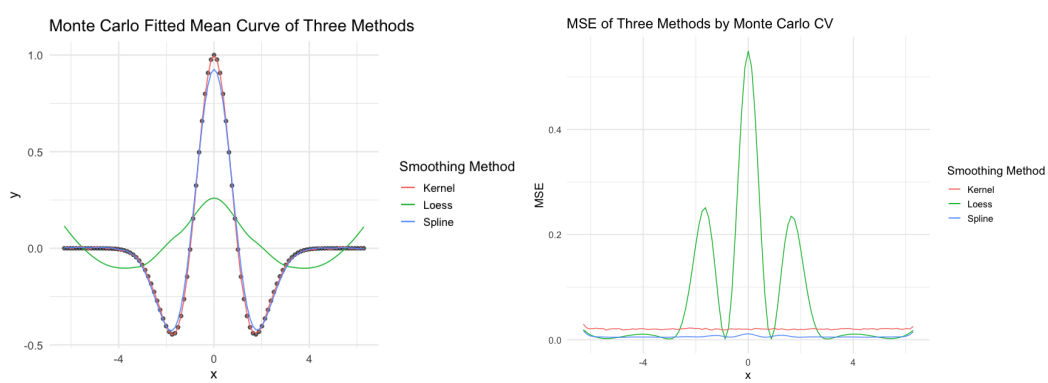

Figure 3—mean fitted curve and MSE plot with equidistant points

1. The Mexican hat function is characterized by a sharp peak at x=0 and valleys near $x = \pm\sqrt{3}$(approximately ± 1.73), with rapid changes in curvature. From the fitted mean curve plot, we can observe that the performance of LOESS (green curve) shows clear underfitting. It fails to capture the sharp peak around x=0 and the valleys near $\pm\sqrt{3}$, instead producing an overly smoothed, wide, and flattened hump, resulting in the largest deviation from the true function. In contrast, both the Kernel (red curve) and Spline (blue curve) methods perform

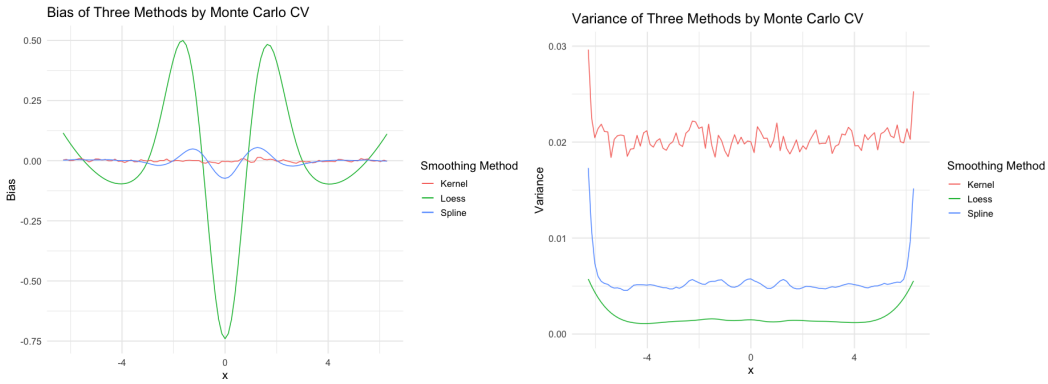

Figure 4—bias and variance plot with equidistant points

well, with fitted curves closely overlapping the true values (dark points). The Spline curve in particular provides an almost perfect fit.

2. In terms of bias, both Kernel smoothing and Spline smoothing exhibit very low bias, with the Kernel method showing an almost zero bias curve across the entire range. In contrast, LOESS shows a much higher bias around -0.75 at $x = 0$ and approximately $+0.5$ near $x = \pm 1.73$. This is primarily due to the relatively large pre-selected span (0.75), which causes the local neighborhood to cover too wide a region. As a result, the model is forced to approximate a highly nonlinear function using overly simple local polynomials, leading to significant model bias. The minimal bias observed in the Kernel and Spline estimators indicates that both methods successfully captured the nonlinear structure of the true function.
3. In terms of variance, LOESS shows the smallest variance, fluctuating around 0.002 across most of the x domain. This low variance is a direct consequence of its large bias. Because the span is large, LOESS uses many neighboring observations for averaging and fitting, resulting in a highly smoothed estimate. Consequently, it becomes less sensitive to random fluctuations across different datasets, leading to very low variance. In contrast, Kernel smoothing exhibits the highest variance, oscillating around 0.02. This is likely due to its relatively small bandwidth, which makes the estimator more sensitive to small perturbations in the training data. Spline smoothing shows a moderate variance level (around 0.005), remaining relatively stable across the entire region. A boundary effect can also be observed: as shown in the variance plots, all three methods tend to have increased variance near the dataset boundaries (around $\pm 2\pi$), especially for

the Spline and Kernel estimators — a common phenomenon in local smoothing methods.

4. For the MSE, since $MSE = Bias^2 + Var$, the high MSE of LOESS is entirely driven by its large bias. Although it has the lowest variance, such a substantial bias renders the model practically useless for accurate prediction. In contrast, Spline smoothing achieves the lowest MSE, followed closely by Kernel smoothing. This indicates that, in this experiment, Spline smoothing attains the best bias–variance trade-off, making it the most effective model among the three.
5. When considering whether this is a fair comparison, the answer is no—it is not entirely fair. This is because Spline smoothing (*smooth.spline()*) automatically selects its optimal smoothing parameter using Generalized Cross-Validation (GCV) when no parameter is manually specified. As a result, it can adaptively find the best bias–variance trade-off based on the data structure. In contrast, Kernel smoothing (*ksmooth()*) was implemented with a fixed bandwidth = 0.2. Although this is a predetermined value, the results show that it happens to be a reasonable choice, as it yields very low bias in this experiment. However, for LOESS (*loess()*), we used span = 0.75, which means that about 75% of the data points are used for each local fit. For a function like the Mexican Hat, which exhibits sharp local peaks and rapid fluctuations, such a large span forces the LOESS curve to be overly smooth and unable to capture the fine local structure—resulting in severe underfitting, high bias, and consequently a very large MSE.
6. The poor performance of LOESS is not due to the method itself, but rather the inappropriate choice of the smoothing parameter (span). If the span were set to a smaller value—such as 0.1, or determined automatically using cross-validation—LOESS would be able to capture more local details and sharp variations in the data. In that case, its performance could potentially be comparable to that of Spline and Kernel smoothing. The experiment designed to test this assumption has been conducted; please refer to the "Results of Extra Experiment" section on page 8 and 9.

4.2 Analyze the results of Experiment 2

1. The fitted mean curve of LOESS shows significant improvement, with results much closer to the true function. This improvement is mainly due to the use of a more suitable smoothing parameter (span = 0.3365). Kernel smoothing (red

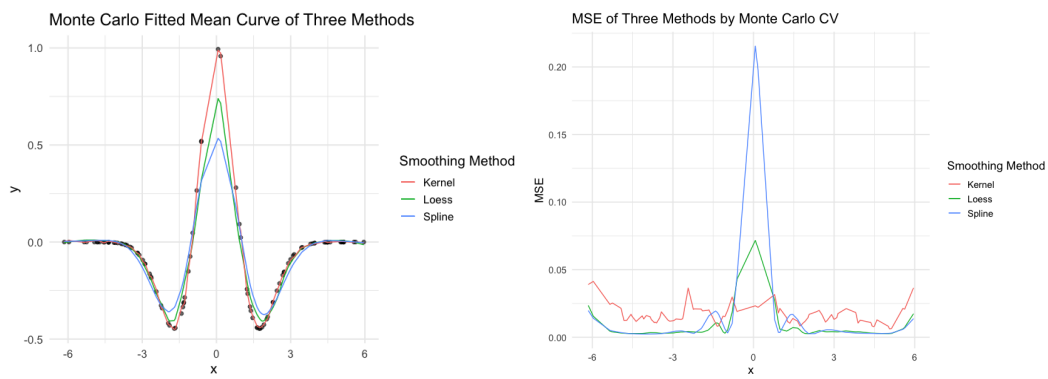

Figure 5—mean fitted curve and MSE plot with non-equidistant points

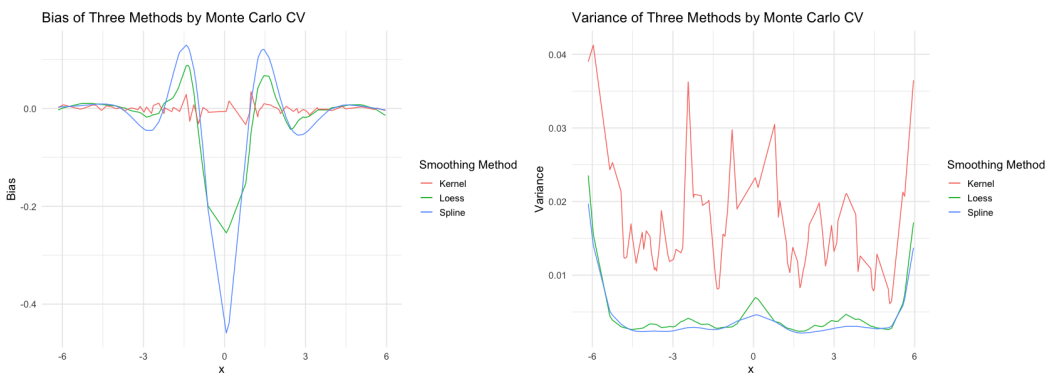

Figure 6—bias and variance plot with non-equidistant points

curve) performs best overall — the fitted curve almost perfectly overlaps with the true curve, especially near x = 0.

2. For bias, Spline smoothing exhibits the largest bias (the peak around −0.45 near x = 0). This is likely because the chosen spar = 0.7163 leads to over-smoothing, preventing the spline from adapting to regions of high curvature. LOESS also shows notable bias, but it remains smaller than that of the spline. Kernel smoothing, on the other hand, has the flattest bias curve—close to zero—indicating that its bandwidth = 0.2 effectively captures the structure of the true function.
3. In terms of variance, kernel smoothing has the highest and most fluctuating variance. This instability arises because with non-equidistant x values, the kernel estimator faces data sparsity in certain regions, resulting in fewer local observations and greater variability. Spline smoothing has the lowest variance—this stability is achieved at the cost of over-smoothing. Its strong regularization constraint (large spar value) sacrifices fitting accuracy for robustness.

4. Spline smoothing yields the highest MSE (peak around 0.22), primarily driven by its high bias. Kernel smoothing performs the best in terms of MSE — although it has the largest variance, its extremely low bias allows it to achieve the best overall bias–variance trade-off.
5. Uneven data density poses significant challenges for local smoothing methods: one is to affect the Local constant fitting stability (NW Kernel). In sparse regions of x, the reduced number of nearby points used in weighted averaging increases the local estimation variance, as evidenced by the large fluctuations in the red curve. The other is to increase the complexity of parameter tuning. When data density is uneven, using a single fixed smoothing parameter (e.g., span = 0.3365 or bandwidth = 0.2) cannot perform well in both dense and sparse regions. Ideally, the bandwidth should adapt to the local data density. In summary, the best practice is to use cross-validation to select the optimal smoothing parameters for each method, ensuring a fair bias–variance balance.
6. For computational challenges, LOESS requires performing local polynomial fitting at every prediction point xi, which is computationally expensive for large, high-dimensional datasets.The Nadaraya–Watson kernel smoothing is more efficient since it only computes weighted averages of existing points without solving equations. Spline smoothing is the most computationally efficient among the three; it solves a penalized least squares problem through a large linear system, and the *smooth.spline()* function in R uses an approximately $O(n)$ algorithm, making it highly efficient even for large datasets.

4.3 Results of Extra Experiment

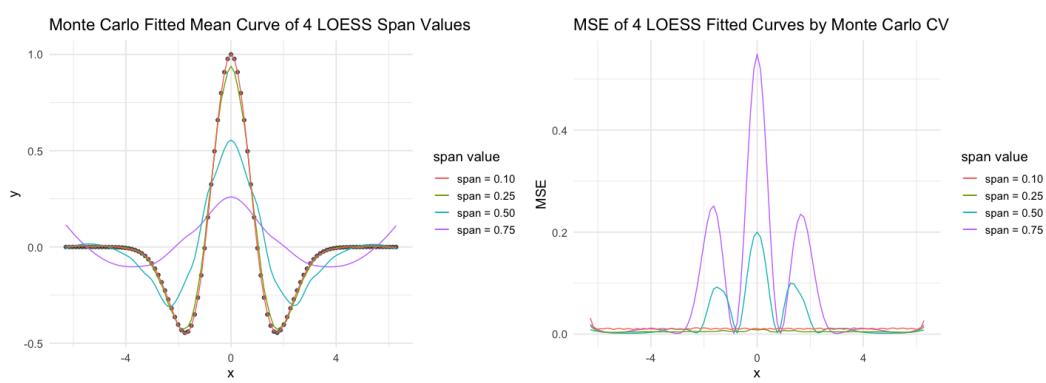

Figure 7—mean fitted LOESS curves and their MSE plots against equidistant points with 4 different smoothing parameters (span)

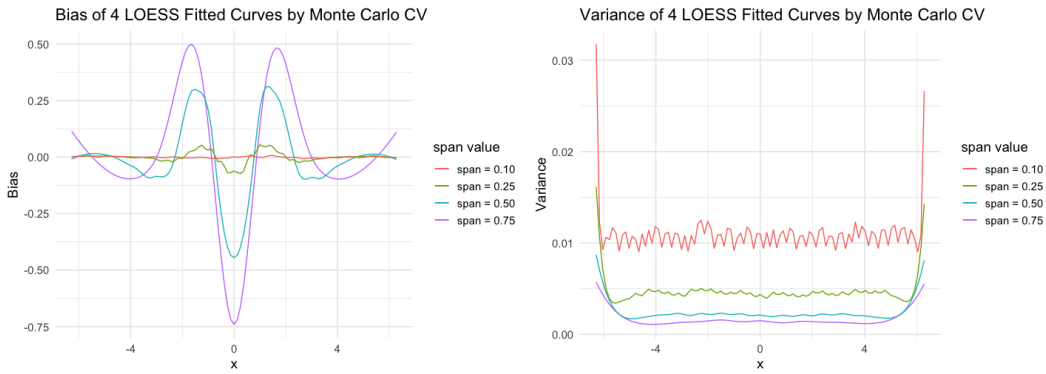

Figure 8—bias and variance plots of LOESS curves against equidistant points with 4 different smoothing parameters (span)

To demonstrate that Experiment 1 was not a fair comparison among the models, we tested four different span values for the LOESS local smoothing method (span = 0.1, 0.25, 0.5, and 0.75) using equidistant points. As the span value decreases from 0.75 to 0.1, the fitted curve improves noticeably. Around the sharp peak at x=0, models with smaller spans (0.25 and 0.10) produce fitted curves that closely overlap the true function. In general, smaller spans (0.25 and 0.10) make the fitted curve more flexible, allowing LOESS to capture the local structure of the Mexican hat function accurately. Consequently, the bias rapidly decreases to nearly zero.

The reduction in bias comes at the cost of increased variance. As the span decreases (from $0.75 \rightarrow 0.10$), the local neighborhood used for fitting becomes smaller, making the model more sensitive to random noise in the data. This leads to a sharp increase in variance (from approximately 0.002 to 0.01, excluding boundary effects near $\pm 2\pi$). However, in this experiment, the reduction in bias is much greater than the increase in variance. As a result, the overall MSE decreases as the span becomes smaller, especially near the sharp peak at x=0. This confirms that for highly curved functions like the Mexican hat, achieving low bias is the key to optimal performance.

This experiment demonstrates that the poor performance of LOESS in Experiment 1 was not due to an inherent weakness of the method, but rather the result of an inappropriately large span (0.75). By selecting a more suitable span value (such as 0.10 or 0.25), LOESS can achieve performance comparable to Spline and Kernel smoothing, showing that all local smoothing methods can be competitive when their parameters are properly tuned.

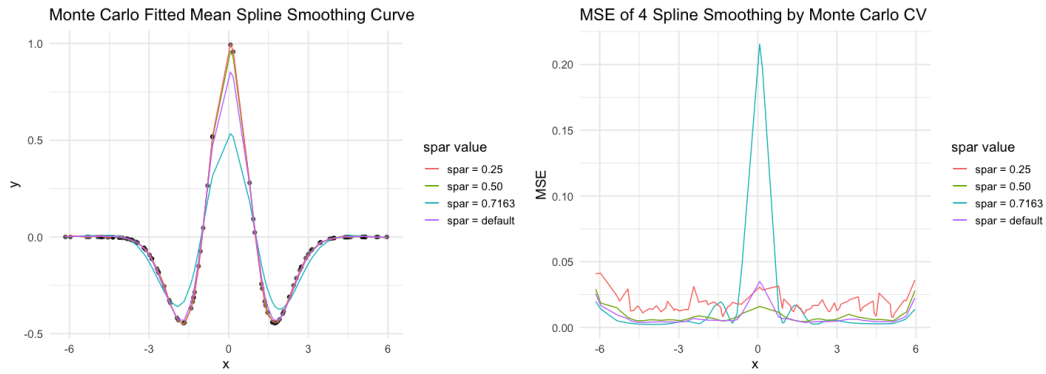

Figure 9—mean fitted spline smoothing and their MSE plots against non-equidistant points with 4 different smoothing parameters (spar)

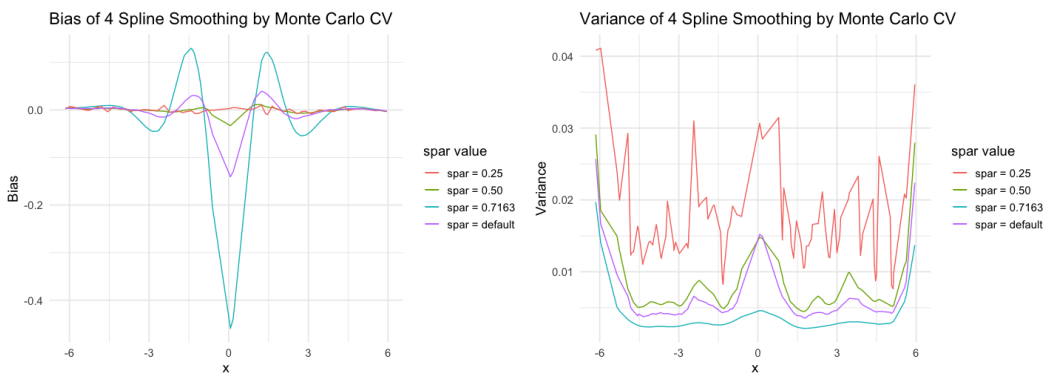

Figure 10—bias and variance plots of spline smoothing against non-equidistant points with 4 different smoothing parameters (spar)

In the second extra experiment, we aimed to verify that the poor performance of spline smoothing with non-equidistant points in Experiment 2 was also a case of unfair comparison. In that experiment, the smoothing parameter was fixed at spar = 0.7163, which was too large for the target function. By comparing results across different values of spar (from 0.25 to 0.7163) as well as the automatically optimized parameter provided by the smooth.spline() function, we observed that spar = 0.7163 yielded the worst fit. When spar was reduced below 0.5, spline smoothing outperformed the best-performing model in Experiment 2 (kernel smoothing), achieving lower MSE and rapidly decreasing bias close to zero while accurately capturing the sharp peak of the Mexican Hat function.

As spar decreased (e.g., to 0.25), the variance increased, but due to the very low bias, the overall MSE remained at a low level. Notably, the default auto-optimized parameter (purple curve) provided the best initial trade-off between bias and variance, showing a stable and low MSE across the entire domain.

This analysis clearly demonstrates that under non-equidistant sampling, when spar is tuned to 0.5 or lower, spline smoothing performs as well as (or better than) the best smoothing method in Experiment 2. This reinforces the importance of using data-driven parameter selection methods (such as cross-validation) when comparing smoothing techniques — otherwise, any comparison risks being unfair or misleading.

5 CONCLUSION

In this homework, we find that local smoothing methods face inherent challenges in accurately estimating the Mexican hat function around x=0 due to its sharp curvature, which exposes the bias–variance trade-off. In the initial equidistant experiment, we were surprised to find that LOESS performed extremely poorly with a span = 0.75; its MSE was entirely dominated by a large bias, making the comparison with the superior Kernel and Spline results unfair. Subsequent sensitivity analyses (varying the span for LOESS and spar for Spline in non-equidistant experiment) confirmed this: when the smoothing parameter was decreased to better adapt to the sharp local structure, the model's bias dropped sharply, while its variance increased accordingly—a clear illustration that a smaller (larger) bias often corresponds to a larger (smaller) variance. Despite this trade-off, the substantial reduction in bias led to a significant improvement in overall MSE. This demonstrates that achieving optimal predictive performance for high-curvature functions requires prioritizing bias reduction through appropriate parameter tuning, ideally guided by data-driven techniques such as cross-validation.

This set of experiments answers the question posed in the introduction regarding method superiority: there is no single best method, only the best-tuned method. In terms of computational efficiency, Spline Smoothing is very efficient due to its nearly O(n) algorithm, while Kernel Smoothing is also quite efficient as it avoids solving large linear systems. LOESS, however, faces the greatest computational challenge on large, high dimensional datasets because it must repeatedly solve weighted least squares regressions for every prediction point. The non-equidistant design further intensified variance fluctuations in data-sparse regions (especially

for the Kernel method), highlighting the importance of parameter selection to prevent undersmoothing in dense areas or oversmoothing in sparse areas. In summary, all three non-parametric smoothers—when properly tuned—can provide competitive, low-MSE estimates for the Mexican hat function, underscoring that fairness in model comparison ultimately depends on consistent and data-driven parameter optimization.

6 APPENDIX (R CODE)

```
library(ggplot2)
library(dplyr)
library(tidyr)

m <- 1000 #Monte Carlo runs: j
n <- 101 #sample size :i
x <- 2*pi*seq(-1, 1, length=n)
y_true <- (1-x^2)*exp(-0.5*x^2)
# EDA plot
ggplot() +
  geom_point(aes(x = x, y = y_true), color = "blue", pch=1,size = 2, alpha = 0.8)

set.seed(7406)
error<-rnorm(length(x), sd=0.2)
ggplot() +
  geom_point(aes(x = x, y = y_true+error), color = "blue", pch=1,size = 2, alpha = 0.8)

#matrix of to store fitted values for three methods
fv_loess <- fv_kernel <- fv_spline <- matrix(0, nrow= n, ncol= m)

##Generate all true y value+error, fit 3 models and store the fitted values
set.seed(7406)
for (j in 1:m){
  ## simulate y-values
  y <- (1-x^2)*exp(-0.5*x^2) + rnorm(length(x), sd=0.2);
  ## Get the estimates and store them
  fv_loess[,j] <- predict(loess(y ~ x, span = 0.75), newdata = x);
  fv_kernel[,j] <- ksmooth(x, y, kernel="normal", bandwidth= 0.2, x.points=x)$y;
  fv_spline[,j] <- predict(smooth.spline(y ~ x), x=x)$y
}
# calculate and plot the mean of three estimators in a single plot
mean_loess <- apply(fv_loess,1,mean)
mean_kernel <- apply(fv_kernel,1,mean)
mean_spline <- apply(fv_spline,1,mean)
```

```r
#dmin <- min( mean_loess, mean_kernel,mean_spline)
#dmax <- max( mean_loess, mean_kernel, mean_spline)

dim(fv_loess) #dim=101*1000

df_meanplot <- data.frame(x = x, True = y_true,Loess = mean_loess,Kernel = mean_kernel,
  Spline = mean_spline)
ggplot() +
  geom_point(aes(x = x, y = y_true), color = "black", size = 1.5, alpha = 0.6) +
  geom_line(data = df_meanplot , aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(data = df_meanplot , aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(data = df_meanplot , aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "Monte Carlo Fitted Mean Curve of Three Methods",x = "x", y = "y",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

# calculate and plot the bias
bias_loess<-mean_loess-y_true
bias_kernel<-mean_kernel-y_true
bias_spline<-mean_spline-y_true

df_biasplot <- data.frame(x = x,Loess = bias_loess,Kernel = bias_kernel,Spline = bias_spline)
df_biasplot %>% ggplot() +
  geom_line(aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "Bias of Three Methods by Monte Carlo CV",x = "x", y = "Bias",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

# calculate and plot the Var
var_loess<-rowMeans((fv_loess - rowMeans(fv_loess))^2)
var_kernel<-rowMeans((fv_kernel - rowMeans(fv_kernel))^2)
var_spline<-rowMeans((fv_spline - rowMeans(fv_spline))^2)

df_varplot <- data.frame(x = x,Loess = var_loess,Kernel = var_kernel,Spline = var_spline)
df_varplot %>% ggplot() +
  geom_line(aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "Variance of Three Methods by Monte Carlo CV",x = "x", y = "Variance",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)
```

```r
# calculate and plot the MSE
mse_loess<-rowMeans((fv_loess - y_true)^2)
mse_kernel<-rowMeans((fv_kernel - y_true)^2)
mse_spline<-rowMeans((fv_spline - y_true)^2)

df_mseplot <- data.frame(x = x,Loess = mse_loess,Kernel = mse_kernel,Spline = mse_spline)
df_mseplot %>% ggplot() +
  geom_line(aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "MSE of Three Methods by Monte Carlo CV",x = "x", y = "MSE",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

####-------Extra Experiment for equidistant dataset--------
#matrix of to store fitted values LOESS with 4 diff span values
fv_loess75 <- fv_loess50 <- fv_loess25 <- fv_loess10 <-matrix(0, nrow= n, ncol= m)

##Generate all true y value+error, fit 3 models and store the fitted values
set.seed(7406)
for (j in 1:m){
  ## simulate y-values
  y <- (1-x^2)*exp(-0.5*x^2) + rnorm(length(x), sd=0.2);
  ## Get the estimates and store them
  fv_loess75[,j] <- predict(loess(y ~ x, span = 0.75), newdata = x);
  fv_loess50[,j] <- predict(loess(y ~ x, span = 0.5), newdata = x);
  fv_loess25[,j] <- predict(loess(y ~ x, span = 0.25), newdata = x);
  fv_loess10[,j] <- predict(loess(y ~ x, span = 0.1), newdata = x);
}
# calculate and plot the mean
mean_loess75 <- apply(fv_loess75,1,mean)
mean_loess50 <- apply(fv_loess50,1,mean)
mean_loess25 <- apply(fv_loess25,1,mean)
mean_loess10 <- apply(fv_loess10,1,mean)

df_meanplotLOESS <- data.frame(x = x, True = y_true,span75 = mean_loess75,
                               span50 = mean_loess50,span25 = mean_loess25,
                               span10 = mean_loess10)
ggplot() +
  geom_point(aes(x = x, y = y_true), color = "black", size = 1.5, alpha = 0.6) +
  geom_line(data = df_meanplotLOESS , aes(x = x, y = span75, color = "span75"), size = 0.5) +
  geom_line(data = df_meanplotLOESS , aes(x = x, y = span50, color = "span50"), size = 0.5) +
  geom_line(data = df_meanplotLOESS , aes(x = x, y = span25, color = "span25"), size = 0.5) +
  geom_line(data = df_meanplotLOESS , aes(x = x, y = span10, color = "span10"), size = 0.5) +
```

```
  labs(title = "Monte Carlo Fitted Mean Curve of 4 LOESS Span Values",x = "x", y = "y",
                              color ="span value") +
  scale_color_discrete(labels = c("span75" = "span = 0.75", "span50" = "span = 0.50",
                                  "span25" = "span = 0.25", "span10" = "span = 0.10"))+
  theme_minimal(base_size = 14)
# calculate and plot the bias
bias_loess75<-mean_loess75-y_true
bias_loess50<-mean_loess50-y_true
bias_loess25<-mean_loess25-y_true
bias_loess10<-mean_loess10-y_true

df_biasplotLOESS <- data.frame(x = x,span75 = bias_loess75,span50 = bias_loess50,
                                span25 = bias_loess25,span10 = bias_loess10)

df_biasplotLOESS  %>% ggplot() +
  geom_line(aes(x = x, y = span75, color = "span75"), size = 0.5) +
  geom_line(aes(x = x, y = span50, color = "span50"), size = 0.5) +
  geom_line(aes(x = x, y = span25, color = "span25"), size = 0.5) +
  geom_line(aes(x = x, y = span10, color = "span10"), size = 0.5) +
  labs(title = "Bias of 4 LOESS Fitted Curves by Monte Carlo CV",x = "x", y = "Bias",
                              color ="span value") +
  scale_color_discrete(labels = c("span75" = "span = 0.75", "span50" = "span = 0.50",
                                  "span25" = "span = 0.25", "span10" = "span = 0.10"))+
  theme_minimal(base_size = 14)
# calculate and plot the Var
var_loess75<-rowMeans((fv_loess75 - rowMeans(fv_loess75))^2)
var_loess50<-rowMeans((fv_loess50 - rowMeans(fv_loess50))^2)
var_loess25<-rowMeans((fv_loess25 - rowMeans(fv_loess25))^2)
var_loess10<-rowMeans((fv_loess10 - rowMeans(fv_loess10))^2)

df_varplotLOESS <- data.frame(x = x, span75=var_loess75,span50 = var_loess50,
                         span25 = var_loess25,span10 = var_loess10)
 df_varplotLOESS %>% ggplot() +
  geom_line(aes(x = x, y = span75, color = "span75"), size = 0.5) +
  geom_line(aes(x = x, y = span50, color = "span50"), size = 0.5) +
  geom_line(aes(x = x, y = span25, color = "span25"), size = 0.5) +
  geom_line(aes(x = x, y = span10, color = "span10"), size = 0.5) +
  labs(title = "Variance of 4 LOESS Fitted Curves by Monte Carlo CV",x = "x",
                              y = "Variance", color ="span value") +
  scale_color_discrete(labels = c("span75" = "span = 0.75", "span50" = "span = 0.50",
                                  "span25" = "span = 0.25", "span10" = "span = 0.10"))+
  theme_minimal(base_size = 14)
# calculate and plot the MSE
mse_loess75<-rowMeans((fv_loess75 - y_true)^2)
```

```r
mse_loess50<-rowMeans((fv_loess50 - y_true)^2)
mse_loess25<-rowMeans((fv_loess25 - y_true)^2)
mse_loess10<-rowMeans((fv_loess10 - y_true)^2)

df_mseplotLOESS <- data.frame(x = x,span75=mse_loess75,span50=mse_loess50,
                              span25=mse_loess25,span10=mse_loess10)
df_mseplotLOESS %>% ggplot() +
  geom_line(aes(x = x, y = span75, color = "span75"), size = 0.5) +
  geom_line(aes(x = x, y = span50, color = "span50"), size = 0.5) +
  geom_line(aes(x = x, y = span25, color = "span25"), size = 0.5) +
  geom_line(aes(x = x, y = span10, color = "span10"), size = 0.5) +
  labs(title = "MSE of 4 LOESS Fitted Curves by Monte Carlo CV",x = "x", y = "MSE",
                              color ="span value") +
  scale_color_discrete(labels = c("span75" = "span = 0.75", "span50" = "span = 0.50",
                                  "span25" = "span = 0.25", "span10" = "span = 0.10"))+
  theme_minimal(base_size = 14)

#Non-equidistant dataset
x <- read.table(file="HW04part2-1.x.csv", sep=",", header=TRUE)[,1] #x is a vector
y_true <- (1-x^2)*exp(-0.5*x^2)
# EDA
ggplot() +
  geom_point(aes(x = x, y = y_true), color = "blue", pch=1,size = 2, alpha = 0.8)

set.seed(7406)
error<-rnorm(length(x), sd=0.2)
ggplot() +
  geom_point(aes(x = x, y = y_true+error), color = "blue", pch=1,size = 2, alpha = 0.8)

#matrix of to store fitted values for three methods
fv_loess <- fv_kernel <- fv_spline <- matrix(0, nrow= n, ncol= m)
##Generate all true y value+error, fit 3 models and store the fitted values
set.seed(7406)
for (j in 1:m){
  ## simulate y-values
  y <- (1-x^2)*exp(-0.5*x^2) + rnorm(length(x), sd=0.2)
  ## Get the estimates and store them
  fv_loess[,j] <- predict(loess(y ~ x, span = 0.3365), newdata = x)
  fv_kernel[,j] <- ksmooth(x, y, kernel="normal", bandwidth= 0.2, x.points=x)$y
  fv_spline[,j] <- predict(smooth.spline(y ~ x,spar=0.7163), x=x)$y
}
# calculate and plot the mean of three estimators in a single plot
mean_loess <- apply(fv_loess,1,mean)
mean_kernel <- apply(fv_kernel,1,mean)
```

```r
mean_spline <- apply(fv_spline,1,mean)

df_meanplot <- data.frame(x = x, True = y_true,Loess = mean_loess,Kernel = mean_kernel,
                          Spline = mean_spline)
ggplot() +
  geom_point(aes(x = x, y = y_true), color = "black", size = 1.5, alpha = 0.6) +
  geom_line(data = df_meanplot , aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(data = df_meanplot , aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(data = df_meanplot , aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "Monte Carlo Fitted Mean Curve of Three Methods",x = "x", y = "y",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

# calculate and plot the bias
bias_loess<-mean_loess-y_true
bias_kernel<-mean_kernel-y_true
bias_spline<-mean_spline-y_true

df_biasplot <- data.frame(x = x,Loess = bias_loess,Kernel = bias_kernel,Spline = bias_spline)
df_biasplot %>% ggplot() +
  geom_line(aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "Bias of Three Methods by Monte Carlo CV",x = "x", y = "Bias",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

# calculate and plot the Var
var_loess<-rowMeans((fv_loess - rowMeans(fv_loess))^2)
var_kernel<-rowMeans((fv_kernel - rowMeans(fv_kernel))^2)
var_spline<-rowMeans((fv_spline - rowMeans(fv_spline))^2)

df_varplot <- data.frame(x = x,Loess = var_loess,Kernel = var_kernel,Spline = var_spline)
df_varplot %>% ggplot() +
  geom_line(aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "Variance of Three Methods by Monte Carlo CV",x = "x", y = "Variance",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

# calculate and plot the MSE
mse_loess<-rowMeans((fv_loess - y_true)^2)
mse_kernel<-rowMeans((fv_kernel - y_true)^2)
```

```r
mse_spline<-rowMeans((fv_spline - y_true)^2)

df_mseplot <- data.frame(x = x,Loess = mse_loess,Kernel = mse_kernel,Spline = mse_spline)
df_mseplot %>% ggplot() +
  geom_line(aes(x = x, y = Loess, color = "Loess"), size = 0.5) +
  geom_line(aes(x = x, y = Kernel, color = "Kernel"), size = 0.5) +
  geom_line(aes(x = x, y = Spline, color = "Spline"), size = 0.5) +
  labs(title = "MSE of Three Methods by Monte Carlo CV",x = "x", y = "MSE",
                              color ="Smoothing Method") +
  theme_minimal(base_size = 14)

####-------Extra Experiment for non-equidistant dataset--------
#matrix of to store fitted values LOESS with 4 diff span values
fv_spline7163 <- fv_spline50 <- fv_spline25 <- fv_spline_df <-matrix(0, nrow= n, ncol= m)

##Generate all true y value+error, fit 3 models and store the fitted values
set.seed(7406)
for (j in 1:m){
  ## simulate y-values
  y <- (1-x^2)*exp(-0.5*x^2) + rnorm(length(x), sd=0.2);
  ## Get the estimates and store them
  fv_spline7163[,j] <- predict(smooth.spline(y ~ x,spar=0.7163), x=x)$y
  fv_spline50[,j] <- predict(smooth.spline(y ~ x,spar=0.5), x=x)$y
  fv_spline25[,j] <- predict(smooth.spline(y ~ x,spar=0.25), x=x)$y
  fv_spline_df[,j] <- predict(smooth.spline(y ~ x), x=x)$y
}
# calculate and plot the mean
mean_spline7163 <- apply(fv_spline7163,1,mean)
mean_spline50 <- apply(fv_spline50,1,mean)
mean_spline25 <- apply(fv_spline25,1,mean)
mean_spline_df <- apply(fv_spline_df,1,mean)

df_meanplotspline <- data.frame(x = x, True = y_true,spar7163 = mean_spline7163,
                                spar50 = mean_spline50,spar25 = mean_spline25,
                                spardf = mean_spline_df)
ggplot() +
  geom_point(aes(x = x, y = y_true), color = "black", size = 1.5, alpha = 0.6) +
  geom_line(data = df_meanplotspline , aes(x = x, y = spar7163, color = "spar7163"), size = 0.5) +
  geom_line(data = df_meanplotspline, aes(x = x, y = spar50, color = "spar50"), size = 0.5) +
  geom_line(data = df_meanplotspline, aes(x = x, y = spar25, color = "spar25"), size = 0.5) +
  geom_line(data = df_meanplotspline, aes(x = x, y = spardf, color = "spardf"), size = 0.5) +
  labs(title = "Monte Carlo Fitted Mean Spline Smoothing Curve",x = "x", y = "y", color ="spar value") +
  scale_color_discrete(labels = c("spar7163" = "spar = 0.7163", "spar50" = "spar = 0.50",
                                  "spar25" = "spar = 0.25", "spardf" = "spar = default"))+
```

```
  theme_minimal(base_size = 14)
# calculate and plot the bias
bias_spline7163<-mean_spline7163-y_true
bias_spline50<-mean_spline50-y_true
bias_spline25<-mean_spline25-y_true
bias_spline_df<-mean_spline_df-y_true

df_biasplotSPLINE <- data.frame(x = x,spar7163 = bias_spline7163,spar50 = bias_spline50,
                                spar25 = bias_spline25,spardf = bias_spline_df)

df_biasplotSPLINE  %>% ggplot() +
  geom_line(aes(x = x, y = spar7163, color = "spar7163"), size = 0.5) +
  geom_line(aes(x = x, y = spar50, color = "spar50"), size = 0.5) +
  geom_line(aes(x = x, y = spar25, color = "spar25"), size = 0.5) +
  geom_line(aes(x = x, y = spardf, color = "spardf"), size = 0.5) +
  labs(title = "Bias of 4 Spline Smoothing by Monte Carlo CV",x = "x", y = "Bias", color ="spar value") +
  scale_color_discrete(labels = c("spar7163" = "spar = 0.7163", "spar50" = "spar = 0.50",
                                  "spar25" = "spar = 0.25", "spardf" = "spar = default"))+
  theme_minimal(base_size = 14)
# calculate and plot the Var
var_spline7163<-rowMeans((fv_spline7163 - rowMeans(fv_spline7163))^2)
var_spline50<-rowMeans((fv_spline50 - rowMeans(fv_spline50))^2)
var_spline25<-rowMeans((fv_spline25 - rowMeans(fv_spline25))^2)
var_spline_df<-rowMeans((fv_spline_df - rowMeans(fv_spline_df))^2)

df_varplotSPLINE <- data.frame(x = x, spar7163=var_spline7163,spar50 = var_spline50,
                               spar25 = var_spline25,spardf = var_spline_df)
df_varplotSPLINE %>% ggplot() +
  geom_line(aes(x = x, y = spar7163, color = "spar7163"), size = 0.5) +
  geom_line(aes(x = x, y = spar50, color = "spar50"), size = 0.5) +
  geom_line(aes(x = x, y = spar25, color = "spar25"), size = 0.5) +
  geom_line(aes(x = x, y = spardf, color = "spardf"), size = 0.5) +
  labs(title = "Variance of 4 Spline Smoothing by Monte Carlo CV",x = "x", y = "Variance",
                               color ="spar value") +
  scale_color_discrete(labels = c("spar7163" = "spar = 0.7163", "spar50" = "spar = 0.50",
                                  "spar25" = "spar = 0.25", "spardf" = "spar = default"))+
  theme_minimal(base_size = 14)

# calculate and plot the MSE
mse_spline7163<-rowMeans((fv_spline7163 - y_true)^2)
mse_spline50<-rowMeans((fv_spline50 - y_true)^2)
mse_spline25<-rowMeans((fv_spline25 - y_true)^2)
mse_spline_df<-rowMeans((fv_spline_df - y_true)^2)
```

```r
df_mseplotSPLINE <- data.frame(x = x,spar7163=mse_spline7163,spar50=mse_spline50,
                               spar25=mse_spline25,spardf=mse_spline_df)
df_mseplotSPLINE %>% ggplot() +
  geom_line(aes(x = x, y = spar7163, color = "spar7163"), size = 0.5) +
  geom_line(aes(x = x, y = spar50, color = "spar50"), size = 0.5) +
  geom_line(aes(x = x, y = spar25, color = "spar25"), size = 0.5) +
  geom_line(aes(x = x, y = spardf, color = "spardf"), size = 0.5) +
  labs(title = "MSE of 4 Spline Smoothing by Monte Carlo CV",x = "x", y = "MSE", color ="spar value") +
  scale_color_discrete(labels = c("spar7163" = "spar = 0.7163", "spar50" = "spar = 0.50",
                                  "spar25" = "spar = 0.25", "spardf" = "spar = default"))+
  theme_minimal(base_size = 14)
```